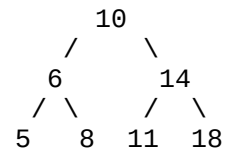


Binary Trees: Part 1

The binary tree is a fundamental data structure used in computer science. The binary tree is a useful data structure for rapidly storing sorted data and rapidly retrieving stored data. A binary tree is composed of parent nodes, or leaves, each of which stores data and also links to up to two other child nodes (leaves) which can be visualized spatially as below the first node with one placed to the left and with one placed to the right. It is the relationship between the leaves linked to and the linking leaf, also known as the parent node, which makes the binary tree such an efficient data structure. It is the leaf on the left which has a lesser key value (ie, the value used to search for a leaf in the tree), and it is the leaf on the right which has an equal or greater key value. As a result, the leaves on the farthest left of the tree have the lowest values, whereas the leaves on the right of the tree have the greatest values. More importantly, as each leaf connects to two other leaves, it is the beginning of a new, smaller, binary tree. Due to this nature, it is possible to easily access and insert data in a binary tree using search and insert functions recursively called on successive leaves.

The typical graphical representation of a binary tree is essentially that of an upside down tree. It begins with a root node, which contains the original key value. The root node has two child nodes; each child node might have its own child nodes. Ideally, the tree would be structured so that it is a perfectly balanced tree, with each node having the same number of child nodes to its left and to its right. A perfectly balanced tree allows for the fastest average insertion of data or retrieval of data. The worst case scenario is a tree in which each node only has one child node, so it becomes as if it were a linked list in terms of speed. The typical representation of a binary tree looks like the following:



The node storing the 10, represented here merely as 10, is the root node, linking to the left and right child nodes, with the left node storing a lower value than the parent node, and the node on the right storing a greater value than the parent node. Notice that if one removed the root node and the right child nodes, that the node storing the value 6 would be the equivalent a new, smaller, binary tree.

The structure of a binary tree makes the insertion and search functions simple to implement using recursion. In fact, the two insertion and search functions are also both very similar. To insert data into a binary tree involves a function searching for an unused node in the proper position in the tree in which to insert the key value. The insert function is generally a recursive function that continues moving down the levels of a binary tree until there is an unused leaf in a position which follows the rules of placing nodes. The rules are that a lower value should be to the left of the node, and a greater or equal value should be to the right. Following the rules, an insert function should check each node to see if it is empty, if so, it would insert the data to be stored along with the key value (in most implementations, an empty node will simply be a NULL pointer from a parent node, so the function would also have to create the node). If the node is filled already, the insert function should check to see if the key value to be inserted is less than the key value of the current node, and if so, the insert function should be recursively called on the left child node, or if the key value to be inserted is greater than or equal to the key value of the current node the insert function should be recursively called on the right child node. The search function works along a similar fashion. It should check to see if the key value of the current node is the value to be searched. If not, it should check to see if the value to be searched for is less than the value of the node, in which case it should be recursively called on the left child node, or if it is greater than the value of the node, it should be recursively called on the right child node. Of course, it is also necessary to check to ensure that the left or right child node actually exists before calling the

function on the node.

Because binary trees have $\log_2 n$ layers, the average search time for a binary tree is $\log_2 n$. To fill an entire binary tree, sorted, takes roughly $\log_2 n * n$. Lets take a look at the necessary code for a simple implementation of a binary tree. First, it is necessary to have a struct, or class, defined as a node.

```
struct node
{
    int key_value;
    node *left;
    node *right;
};
```

The struct has the ability to store the key_value and contains the two child nodes which define the node as part of a tree. In fact, the node itself is very similar to the node in a linked list. A basic knowledge of the code for a linked list will be very helpful in understanding the techniques of binary trees. Essentially, pointers are necessary to allow the arbitrary creation of new nodes in the tree.

It is most logical to create a binary tree class to encapsulate the workings of the tree into a single area, and also making it reusable. The class will contain functions to insert data into the tree and to search for data. Due to the use of pointers, it will be necessary to include a function to delete the tree in order to conserve memory after the program has finished.

```
class btree
{
    node *root;
    btree();
    ~btree();
    void destroy_tree(node *leaf);
    void insert(int key, node *leaf);
    node *search(int key, node *leaf);
public:
    void insert(int key);
    node *search(int key);
    void destroy_tree();
};
```

The insert and search functions that are public members of the class are designed to allow the user of the class to use the class without dealing with the underlying design. The insert and search functions which will be called recursively are the ones which contain two parameters, allowing them to travel down the tree. The destroy_tree function without arguments is a front for the destroy_tree function which will recursively destroy the tree, node by node, from the bottom up.

The code for the class would look similar to the following:

```
btree::btree()
{
    root=NULL;
}
```

It is necessary to initialize root to NULL for the later functions to be able to recognize that it does not exist.

```
btree::~~btree()
{
```

```

    destroy_tree();
}

```

The `destroy_tree` function will set off the recursive function `destroy_tree` shown below which will actually delete all nodes of the tree.

```

void destroy_tree(node *leaf)
{
    if(leaf!=NULL)
    {
        destroy_tree(leaf->left);
        destroy_tree(leaf->right);
        delete leaf;
    }
}

```

The function `destroy_tree` goes to the bottom of each part of the tree, that is, searching while there is a non-null node, deletes that leaf, and then it works its way back up. The function deletes the leftmost node, then the right child node from the leftmost node's parent node, then it deletes the parent node, then works its way back to deleting the other child node of the parent of the node it just deleted, and it continues this deletion working its way up to the node of the tree upon which `delete_tree` was originally called. In the example tree above, the order of deletion of nodes would be 5 8 6 11 18 14 10. Note that it is necessary to delete all the child nodes to avoid wasting memory.

```

void btree::insert(int key, node *leaf)
{
    if(key< leaf->key_value)
    {
        if(leaf->left!=NULL)
            insert(key, leaf->left);
        else
        {
            leaf->left=new node;
            leaf->left->key_value=key;
            leaf->left->left=NULL;    //Sets the left child of the child node to null
            leaf->left->right=NULL;   //Sets the right child of the child node to null
        }
    }
    else if(key>=leaf->key_value)
    {
        if(leaf->right!=NULL)
            insert(key, leaf->right);
        else
        {
            leaf->right=new node;
            leaf->right->key_value=key;
            leaf->right->left=NULL;   //Sets the left child of the child node to null
            leaf->right->right=NULL;  //Sets the right child of the child node to null
        }
    }
}

```

The case where the root node is still NULL will be taken care of by the insert function that is nonrecursive and available to non-members of the class. The insert function searches, moving down the tree of children nodes, following the prescribed rules, left for a lower value to be inserted and right for a greater value, until it finds an empty node which it creates using the 'new' keyword and initializes with

the key value while setting the new node's child node pointers to NULL. After creating the new node, the insert function will no longer call itself.

```
node *btree::search(int key, node *leaf)
{
    if(leaf!=NULL)
    {
        if(key==leaf->key_value)
            return leaf;
        if(key<leaf->key_value)
            return search(key, leaf->left);
        else
            return search(key, leaf->right);
    }
    else return NULL;
}
```

The search function shown above recursively moves down the tree until it either reaches a node with a key value equal to the value for which the function is searching or until the function reaches an uninitialized node, meaning that the value being searched for is not stored in the binary tree. It returns a pointer to the node to the previous instance of the function which called it, handing the pointer back up to the search function accessible outside the class.

```
void btree::insert(int key)
{
    if(root!=NULL)
        insert(key, root);
    else
    {
        root=new node;
        root->key_value=key;
        root->left=NULL;
        root->right=NULL;
    }
}
```

The public version of the insert function takes care of the case where the root has not been initialized by allocating the memory for it and setting both child nodes to NULL and setting the key_value to the value to be inserted. If the root node already exists, insert is called with the root node as the initial node of the function, and the recursive insert function takes over.

```
node *btree::search(int key)
{
    return search(key, root);
}
```

The public version of the search function is used to set off the search recursion at the root node, keeping it from being necessary for the user to have access to the root node.

```
void btree::destroy_tree()
{
    destroy_tree(root);
}
```

The public version of the destroy tree function is merely used to initialize the recursive destroy_tree

function which then deletes all the nodes of the tree.